

```

/*
 * =====
 * Joint-Track Knee Rehab Sleeve — ESP32 Firmware
 * Team: Aditya Gujar, Aditya Shukla, Arushi Tiwari, Hamza Raza Khan
 * VIT Vellore — Embedded Systems & IoT Project
 * =====
 *
 * Sensors : MPU6050 ×2 (I2C), Flex (GPIO34), FSR402 (GPIO35),
 *           MyoWare EMG (GPIO33), DS18B20 (GPIO4), MAX30102 (I2C)
 * Actuators: Buzzer (GPIO25), LED (GPIO27), Haptic Motor (GPIO26)
 *
 * Required Libraries (add to Wokwi libraries.txt or Arduino IDE):
 * - MPU6050 by Electronic Cats (or I2Cdevlib MPU6050)
 * - OneWire by Paul Stoffregen
 * - DallasTemperature by Miles Burton
 * - SparkFun MAX3010x Pulse and Proximity Sensor Library
 * - ArduinoJson by Benoit Blanchon
 * =====
 */

#include <Wire.h>
#include <math.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ArduinoJson.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1 // No reset pin

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ——— PIN DEFINITIONS ———
#define PIN_BUZZER 25
#define PIN_LED 27
#define PIN_MOTOR 26
#define PIN_FLEX 34 // ADC1_CH6
#define PIN_FORCE 35 // ADC1_CH7
#define PIN_EMG 33 // ADC1_CH5

```

```

#define PIN_TEMP_DATA    4 // DS18B20 OneWire data
#define PIN_SDA          21
#define PIN_SCL          22

// ——— I2C ADDRESSES ———
#define IMU_UPPER_ADDR   0x68 // AD0 = GND
#define IMU_LOWER_ADDR   0x69 // AD0 = 3.3V

// ——— REHAB THRESHOLDS ———
#define TARGET_ANGLE     90 // Target knee flexion in degrees
#define ANGLE_TOLERANCE   5 // ±5° window for "correct"
#define MAX_SAFE_ANGLE   130 // Hard over-bend limit
#define MIN_REP_FLEX      80 // Angle to count as "flexed" for rep
#define MIN_REP_EXTEND    15 // Angle to count as "extended" for rep
#define HOLD_TARGET_MS    3000 // 3-second hold target

#define TEMP_INFLAM_THRESH 38.5f // °C inflammation threshold
#define FORCE_OVERLOAD     3000 // ADC raw threshold for excessive load
#define EMG_FATIGUE_THRESH 2800 // ADC raw threshold for muscle fatigue
#define HR_MAX_BPM        130 // Heart rate ceiling during exercise

// ——— TIMING ———
#define TEMP_POLL_INTERVAL 10000UL // Poll DS18B20 every 10s
#define HR_POLL_INTERVAL   5000UL // Poll MAX30102 every 5s
#define SERIAL_SEND_INTERVAL 500UL // Send JSON every 500ms
#define IMU_SAMPLE_MS      10UL // IMU loop at 100Hz
#define COMP_ALPHA          0.96f // Complementary filter alpha

// ——— MPU6050 REGISTER MAP ———
#define MPU_PWR_MGMT_1      0x6B
#define MPU_ACCEL_XOUT_H    0x3B
#define MPU_GYRO_XOUT_H     0x43
#define MPU_CONFIG          0x1A
#define MPU_GYRO_CONFIG     0x1B
#define MPU_ACCEL_CONFIG    0x1C
#define ACCEL_SCALE         16384.0f // ±2g
#define GYRO_SCALE          131.0f // ±250°/s

// ——— STATE MACHINE ———
enum ExerciseState {

```

```

STATE_IDLE,
STATE_EXTENDED,
STATE_FLEXING,
STATE_AT_TARGET,
STATE_HOLDING,
STATE_RETURNING,
STATE_OVERBEND_ALERT,
STATE_OVERLOAD_ALERT
};

// ——— IMU DATA STRUCTURE —————
struct IMUData {
    float ax, ay, az;
    float gx, gy, gz;
    float pitch; // Computed pitch angle in degrees
    float roll;
};

// ——— GLOBAL STATE —————
IMUData imuUpper, imuLower;

float kneeAngle    = 0.0f;
float flexAngle    = 0.0f; // Flex sensor corroborated angle
float fusedAngle   = 0.0f; // Final weighted fusion

int  repCount      = 0;
int  setCount      = 0;
int  repsPerSet    = 10; // Configurable via Serial command
float holdDuration = 0.0f; // Seconds held at target this rep

float kneeTempC    = 36.5f;
int  heartRateBPM  = 72;
int  forceRaw      = 0;
int  emgRaw        = 0;

ExerciseState exerciseState = STATE_IDLE;

unsigned long lastIMUTime    = 0;
unsigned long lastTempTime   = 0;
unsigned long lastHRTTime    = 0;
unsigned long lastSerialTime = 0;

```

```

unsigned long holdStartTime = 0;
unsigned long stateEntryTime = 0;
unsigned long sessionStartTime = 0;

bool sessionActive = false;
bool inFlexPhase = false;
bool holdCounted = false;
bool inflammWarned = false;
bool buzzLocked = false; // Debounce buzzer to prevent spam
unsigned long buzzLockTime = 0;

// ——— COMPLEMENTARY FILTER STATE ———
float upperPitchFiltered = 0.0f;
float lowerPitchFiltered = 0.0f;
bool imuInitialized = false;

// ——— FLEX SENSOR CALIBRATION (calibrate per device) ———
const int FLEX_ADC_STRAIGHT = 2200; // ADC reading when leg straight
const int FLEX_ADC_BENT = 900; // ADC reading at ~130° bend
// Map: ADC → angle (linear interpolation)

// ——— FUNCTION DECLARATIONS ———
void initMPU(uint8_t addr);
void readMPU(uint8_t addr, IMUData &data);
void updateComplementaryFilter(float &filtered, IMUData &data, float dt);
float flexADCtoAngle(int adcVal);
void updateKneeAngle(float dt);
void runStateMachine();
void triggerFeedback(ExerciseState state);
void buzz(int times, int onMs = 150, int offMs = 80);
void setBuzzLock(int ms);
bool isBuzzReady();
void setLED(bool green);
void hapticPulse(int times, int onMs = 200, int offMs = 100);
void pollTemperature();
void pollHeartRate();
void sendTelemetry();
void processSerialCommand();
String stateToString(ExerciseState s);

```

```
// ----- DS18B20 -----  
  
OneWire      oneWire(PIN_TEMP_DATA);  
DallasTemperature tempSensor(&oneWire);  
  
  
// ----- SETUP -----  
  
void setup() {  
    Serial.begin(115200);  
    Wire.begin(PIN_SDA, PIN_SCL);  
    Wire.setClock(400000); // Fast I2C  
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {  
        Serial.println(F("{\"error\":\"OLED init failed\"}"));  
    }  
    display.clearDisplay();  
    display.setTextSize(1);  
    display.setTextColor(SSD1306_WHITE);  
    display.setCursor(0, 0);  
    display.println(F("JointTrack v1.0"));  
    display.println(F("Ready..."));  
    display.display();  
    // Actuator pins  
    pinMode(PIN_BUZZER, OUTPUT);  
    pinMode(PIN_LED, OUTPUT);  
    pinMode(PIN_MOTOR, OUTPUT);  
    digitalWrite(PIN_BUZZER, LOW);  
    digitalWrite(PIN_LED, LOW);  
    digitalWrite(PIN_MOTOR, LOW);  
  
    // Analog pins (input-only by default on ESP32, no pinMode needed for ADC)  
    // GPIO 34, 35, 33 are input-only pins — safe for ADC  
  
    // IMU init  
    initMPU(IMU_UPPER_ADDR);  
    delay(50);  
    initMPU(IMU_LOWER_ADDR);  
    delay(50);  
  
    // Temperature sensor  
    tempSensor.begin();  
    tempSensor.setResolution(12); // 12-bit = 0.0625°C resolution
```

```

// Startup tone
buzz(2, 100, 60);
delay(200);
buzz(1, 300, 0);

sessionStartTime = millis();
lastIMUTime      = millis();

Serial.println(F("{\"event\":\"boot\",\"device\":\"JointTrack-v1.0\"}"));
}

// ——— MAIN LOOP ———
void loop() {
    unsigned long now = millis();

    // — 100Hz IMU loop —
    if (now - lastIMUTime >= IMU_SAMPLE_MS) {
        float dt = (now - lastIMUTime) / 1000.0f;
        lastIMUTime = now;

        readMPU(IMU_UPPER_ADDR, imuUpper);
        readMPU(IMU_LOWER_ADDR, imuLower);
        updateKneeAngle(dt);
    }

    // — State machine & feedback —
    runStateMachine();

    // — Read analog sensors —
    forceRaw = analogRead(PIN_FORCE);
    emgRaw   = analogRead(PIN_EMG);

    // — 10s temperature poll —
    if (now - lastTempTime >= TEMP_POLL_INTERVAL) {
        lastTempTime = now;
        pollTemperature();
    }

    // — 500ms serial telemetry —
    if (now - lastSerialTime >= SERIAL_SEND_INTERVAL) {

```

```
    lastSerialTime = now;
    sendTelemetry();
    updateDisplay();
}

// — Serial command handler —————
if (Serial.available()) {
    processSerialCommand();
}
}

void updateDisplay() {
    display.clearDisplay();

    // — Row 1: Angle (big text) —————
    display.setTextSize(2);
    display.setCursor(0, 0);
    display.print(fusedAngle, 1);
    display.setTextSize(1);
    display.print(F(" deg"));

    // — Row 2: State —————
    display.setTextSize(1);
    display.setCursor(0, 20);
    display.print(F("State: "));
    display.print(stateToString(exerciseState));

    // — Row 3: Reps and Sets —————
    display.setCursor(0, 32);
    display.print(F("Reps: "));
    display.print(repCount);
    display.print(F(" Sets: "));
    display.print(setCount);

    // — Row 4: Hold timer —————
    display.setCursor(0, 42);
    display.print(F("Hold: "));
    display.print(holdDuration, 1);
    display.print(F("s / 3.0s"));
```

```

// — Row 5: Temp and HR —————

display.setCursor(0, 52);
display.print(kneeTempC, 1);
display.print(F("C  HR:"));
display.print(heartRateBPM);
display.print(F("bpm"));

// — Alert banner (overrides top area) ———

if (exerciseState == STATE_OVERBEND_ALERT) {
    display.fillRect(0, 0, 128, 16, SSD1306_WHITE);
    display.setTextColor(SSD1306_BLACK);
    display.setTextSize(2);
    display.setCursor(4, 0);
    display.print(F("OVERBEND!"));
    display.setTextColor(SSD1306_WHITE);
}
else if (exerciseState == STATE_OVERLOAD_ALERT) {
    display.fillRect(0, 0, 128, 16, SSD1306_WHITE);
    display.setTextColor(SSD1306_BLACK);
    display.setTextSize(2);
    display.setCursor(4, 0);
    display.print(F("OVERLOAD!"));
    display.setTextColor(SSD1306_WHITE);
}

display.display();
}

// ——— MPU6050: INITIALIZE —————

void initMPU(uint8_t addr) {
    // Wake up
    Wire.beginTransmission(addr);
    Wire.write(MPU_PWR_MGMT_1);
    Wire.write(0x00); // Clear sleep bit
    Wire.endTransmission(true);
    delay(10);

    // DLPF: ~44Hz bandwidth (reduces noise)
    Wire.beginTransmission(addr);
    Wire.write(MPU_CONFIG);

```

```

Wire.write(0x03);
Wire.endTransmission(true);

// Gyro: ±250°/s
Wire.beginTransmission(addr);
Wire.write(MPU_GYRO_CONFIG);
Wire.write(0x00);
Wire.endTransmission(true);

// Accel: ±2g
Wire.beginTransmission(addr);
Wire.write(MPU_ACCEL_CONFIG);
Wire.write(0x00);
Wire.endTransmission(true);
}

// ——— MPU6050: READ RAW + COMPUTE PITCH/ROLL ———
void readMPU(uint8_t addr, IMUData &data) {
    Wire.beginTransmission(addr);
    Wire.write(MPU_ACCEL_XOUT_H);
    Wire.endTransmission(false);
    Wire.requestFrom(addr, (uint8_t)14, (uint8_t)true);

    if (Wire.available() < 14) return;

    int16_t rawAx = (Wire.read() << 8) | Wire.read();
    int16_t rawAy = (Wire.read() << 8) | Wire.read();
    int16_t rawAz = (Wire.read() << 8) | Wire.read();
    Wire.read(); Wire.read(); // Temperature bytes — skip
    int16_t rawGx = (Wire.read() << 8) | Wire.read();
    int16_t rawGy = (Wire.read() << 8) | Wire.read();
    int16_t rawGz = (Wire.read() << 8) | Wire.read();

    data.ax = rawAx / ACCEL_SCALE;
    data.ay = rawAy / ACCEL_SCALE;
    data.az = rawAz / ACCEL_SCALE;
    data.gx = rawGx / GYRO_SCALE;
    data.gy = rawGy / GYRO_SCALE;
    data.gz = rawGz / GYRO_SCALE;
}

```

```

// Accelerometer pitch (degrees) — prone to noise but no drift
data.pitch = atan2(-data.ax, sqrt(data.ay * data.ay + data.az * data.az)) * 180.0f / M_PI;
data.roll = atan2(data.ay, data.az) * 180.0f / M_PI;
}

```

```

// ——— COMPLEMENTARY FILTER —————

```

```

/*
 * Fuses accelerometer pitch (no drift, noisy) with gyroscope
 * integration (smooth, drifts over time).
 * alpha = 0.96 → 96% gyro weight, 4% accel correction.
 * Tunable: lower alpha = faster correction, more noise.
 */

```

```

void updateComplementaryFilter(float &filtered, IMUData &data, float dt) {
    // Gyro integration: filtered += gyro_rate * dt
    filtered = COMP_ALPHA * (filtered + data.gy * dt)
        + (1.0f - COMP_ALPHA) * data.pitch;
}

```

```

// ——— KNEE ANGLE CALCULATION —————

```

```

void updateKneeAngle(float dt) {
    if (!imuInitialized) {
        // Seed filters from accelerometer on first run
        upperPitchFiltered = imuUpper.pitch;
        lowerPitchFiltered = imuLower.pitch;
        imuInitialized = true;
        return;
    }

    updateComplementaryFilter(upperPitchFiltered, imuUpper, dt);
    updateComplementaryFilter(lowerPitchFiltered, imuLower, dt);
}

```

```

// Knee angle = difference between thigh and shin segment orientations
kneeAngle = fabs(upperPitchFiltered - lowerPitchFiltered);

```

```

// Flex sensor as secondary estimate
int flexRaw = analogRead(PIN_FLEX);
flexAngle = flexADCtoAngle(flexRaw);

```

```

// Weighted sensor fusion: 70% IMU, 30% flex
// (Flex is noisier at extremes but good mid-range)

```

```

fusedAngle = 0.70f * kneeAngle + 0.30f * flexAngle;

// Clamp to physiological range
fusedAngle = constrain(fusedAngle, 0.0f, 180.0f);
}

// ——— FLEX SENSOR: ADC → ANGLE ———
float flexADCtoAngle(int adcVal) {
    // Linear map from calibrated ADC range to 0°–130°
    adcVal = constrain(adcVal, FLEX_ADC_BENT, FLEX_ADC_STRAIGHT);
    return map(adcVal, FLEX_ADC_STRAIGHT, FLEX_ADC_BENT, 0, 130);
}

// ——— STATE MACHINE ———
/*
 * STATE_IDLE      : Waiting for session start command
 * STATE_EXTENDED   : Leg is straight (angle < MIN_REP_EXTEND)
 * STATE_FLEXING    : Leg bending toward target
 * STATE_AT_TARGET  : Within ±TOLERANCE of TARGET_ANGLE → start hold timer
 * STATE_HOLDING    : Holding target angle for HOLD_TARGET_MS
 * STATE_RETURNING  : Returning to extension after hold
 * STATE_OVERBEND_ALERT: Angle > MAX_SAFE_ANGLE
 * STATE_OVERLOAD_ALERT: Force sensor > FORCE_OVERLOAD
 */
void runStateMachine() {
    if (!sessionActive) return;

    unsigned long now = millis();
    ExerciseState prevState = exerciseState;

    bool atTarget = (fabs(fusedAngle - TARGET_ANGLE) <= ANGLE_TOLERANCE);
    bool overBent = (fusedAngle > MAX_SAFE_ANGLE);
    bool overLoaded = (forceRaw > FORCE_OVERLOAD);
    bool isExtended = (fusedAngle < MIN_REP_EXTEND);
    bool isFlexed = (fusedAngle > MIN_REP_FLEX);

    // Safety overrides — highest priority
    if (overLoaded && exerciseState != STATE_OVERLOAD_ALERT) {
        exerciseState = STATE_OVERLOAD_ALERT;
        stateEntryTime = now;
    }
}

```

```

} else if (overBent && exerciseState != STATE_OVERBEND_ALERT) {
    exerciseState = STATE_OVERBEND_ALERT;
    stateEntryTime = now;
} else {
    switch (exerciseState) {
        case STATE_EXTENDED:
            if (isFlexed) {
                exerciseState = STATE_FLEXING;
                stateEntryTime = now;
            }
            break;

        case STATE_FLEXING:
            if (atTarget) {
                exerciseState = STATE_AT_TARGET;
                stateEntryTime = now;
                holdStartTime = now;
                holdCounted = false;
            } else if (isExtended) {
                // Returned without reaching target — no rep counted
                exerciseState = STATE_EXTENDED;
            }
            break;

        case STATE_AT_TARGET:
            if (!atTarget && !holdCounted) {
                exerciseState = fusedAngle < TARGET_ANGLE ? STATE_RETURNING : STATE_FLEXING;
            } else if (atTarget) {
                exerciseState = STATE_HOLDING;
                stateEntryTime = now;
            }
            break;

        case STATE_HOLDING:
            holdDuration = (now - holdStartTime) / 1000.0f;
            if (!atTarget) {
                // Left target zone — count hold if >= threshold
                if ((now - holdStartTime) >= HOLD_TARGET_MS && !holdCounted) {
                    holdCounted = true;
                }
            }
    }
}

```

```

        exerciseState = STATE_RETURNING;
    }
    break;

case STATE_RETURNING:
    if (isExtended) {
        // Completed full rep
        repCount++;
        exerciseState = STATE_EXTENDED;
        if (repCount % repsPerSet == 0) {
            setCount++;
            buzz(3, 200, 100); // Set complete — triple beep
        }
    } else if (atTarget) {
        exerciseState = STATE_AT_TARGET;
        holdStartTime = now;
        holdCounted = false;
    }
    break;

case STATE_OVERBEND_ALERT:
    if (!overBent && (now - stateEntryTime > 500)) {
        exerciseState = STATE_FLEXING;
    }
    break;

case STATE_OVERLOAD_ALERT:
    if (!overLoaded && (now - stateEntryTime > 500)) {
        exerciseState = STATE_EXTENDED;
    }
    break;

default:
    exerciseState = STATE_EXTENDED;
    break;
}

// Trigger feedback only on state change
if (exerciseState != prevState) {

```

```
    triggerFeedback(exerciseState);
}
}

// ----- FEEDBACK TRIGGER -----
void triggerFeedback(ExerciseState state) {
    switch (state) {
        case STATE_AT_TARGET:
        case STATE_HOLDING:
            // Green LED + single beep + single haptic
            setLED(true);
            if (isBuzzReady()) { buzz(1, 120, 0); setBuzzLock(400); }
            hapticPulse(1, 200, 0);
            break;

        case STATE_OVERBEND_ALERT:
            // Red LED + double beep + double haptic
            setLED(false);
            if (isBuzzReady()) { buzz(2, 200, 100); setBuzzLock(1000); }
            hapticPulse(2, 150, 80);
            break;

        case STATE_OVERLOAD_ALERT:
            // Red LED + rapid triple beep
            setLED(false);
            if (isBuzzReady()) { buzz(3, 100, 60); setBuzzLock(1000); }
            hapticPulse(3, 100, 50);
            break;

        case STATE_EXTENDED:
            setLED(false);
            break;

        case STATE_FLEXING:
            setLED(false);
            break;

        case STATE_RETURNING:
            setLED(false);
            break;
    }
}
```

```

    default:
        break;
    }
}

// ——— BUZZER UTILITY ———
void buzz(int times, int onMs, int offMs) {
    for (int i = 0; i < times; i++) {
        digitalWrite(PIN_BUZZER, HIGH);
        delay(onMs);
        digitalWrite(PIN_BUZZER, LOW);
        if (i < times - 1) delay(offMs);
    }
}

void setBuzzLock(int ms) {
    buzzLocked = true;
    buzzLockTime = millis() + ms;
}

bool isBuzzReady() {
    if (buzzLocked && millis() > buzzLockTime) {
        buzzLocked = false;
    }
    return !buzzLocked;
}

// ——— LED UTILITY ———
void setLED(bool green) {
    // With two-LED wiring: green on GPIO27, red would be GPIO28
    // For single LED: green = HIGH, else = LOW
    digitalWrite(PIN_LED, green ? HIGH : LOW);
}

// ——— HAPTIC MOTOR UTILITY ———
void hapticPulse(int times, int onMs, int offMs) {
    for (int i = 0; i < times; i++) {
        digitalWrite(PIN_MOTOR, HIGH);
        delay(onMs);
    }
}

```

```

    digitalWrite(PIN_MOTOR, LOW);
    if (i < times - 1) delay(offMs);
}
}

// ——— TEMPERATURE POLL ———
void pollTemperature() {
    tempSensor.requestTemperatures();
    float t = tempSensor.getTempCByIndex(0);
    if (t != DEVICE_DISCONNECTED_C) {
        kneeTempC = t;
    }
    // Inflammation warning: only fire once until it cools
    if (kneeTempC > TEMP_INFLAM_THRESH && !inflammWarned) {
        inflammWarned = true;
        if (isBuzzReady()) { buzz(3, 100, 60); setBuzzLock(2000); }
        // Flash LED 3 times
        for (int i = 0; i < 3; i++) {
            setLED(false); delay(150);
            setLED(true); delay(150);
        }
        setLED(false);
    } else if (kneeTempC <= TEMP_INFLAM_THRESH) {
        inflammWarned = false;
    }
}

// ——— HEART RATE POLL ———
/*
 * MAX30102 simplified polling.
 * In Wokwi simulation, this returns a fixed simulated value.
 * Full implementation uses SparkFun MAX3010x library with
 * heart rate averaging over 4 samples.
 */
void pollHeartRate() {
    // Placeholder: In production, replace with MAX30105 library calls
    // heartRateBPM = particleSensor.getHeartRate();
    // For simulation, heartRateBPM stays at default or can be set via Serial
}

```

```
// — SERIAL TELEMETRY —
/*
 * Sends a JSON packet every 500ms over Serial.
 * The web dashboard reads this via Web Serial API.
 *
 * Format:
 * {
 *   "t": <millis>,
 *   "angle": <fused_angle>,
 *   "imuU": <upper_pitch>,
 *   "imuL": <lower_pitch>,
 *   "flex": <flex_angle>,
 *   "force": <force_raw>,
 *   "emg": <emg_raw>,
 *   "temp": <knee_temp>,
 *   "hr": <heart_rate>,
 *   "reps": <rep_count>,
 *   "sets": <set_count>,
 *   "hold": <hold_duration>,
 *   "state": "<state_string>",
 *   "session": <session_active>
 * }
 */
void sendTelemetry() {
  StaticJsonDocument<256> doc;
  doc["t"] = millis();
  doc["angle"] = roundf(fusedAngle * 10) / 10.0f; // 1 decimal
  doc["imuU"] = roundf(upperPitchFiltered * 10) / 10.0f;
  doc["imuL"] = roundf(lowerPitchFiltered * 10) / 10.0f;
  doc["flex"] = roundf(flexAngle);
  doc["force"] = forceRaw;
  doc["emg"] = emgRaw;
  doc["temp"] = roundf(kneeTempC * 10) / 10.0f;
  doc["hr"] = heartRateBPM;
  doc["reps"] = repCount;
  doc["sets"] = setCount;
  doc["hold"] = roundf(holdDuration * 10) / 10.0f;
  doc["state"] = stateToString(exerciseState);
  doc["session"] = sessionActive;
}
```

```

serializeJson(doc, Serial);

Serial.println(); // Newline delimiter for parser
}

// ——— SERIAL COMMAND HANDLER ———
/*
 * Commands (send as JSON or plain strings):
 * START      → start session
 * STOP       → stop session
 * RESET      → reset reps/sets counters
 * TARGET:<angle> → set new target angle (e.g. TARGET:60)
 * SETS:<n>     → set reps per set (e.g. SETS:12)
 * HR:<bpm>     → inject simulated heart rate
 */
void processSerialCommand() {
  String cmd = Serial.readStringUntil("\n");
  cmd.trim();

  if (cmd == "START") {
    sessionActive = true;
    exerciseState = STATE_EXTENDED;
    sessionStartTime = millis();
    repCount = 0; setCount = 0;
    buzz(1, 300, 0);
    Serial.println(F("{\"event\":\"session_start\"}"));
  }
  else if (cmd == "STOP") {
    sessionActive = false;
    exerciseState = STATE_IDLE;
    buzz(2, 200, 100);
    Serial.println(F("{\"event\":\"session_stop\"}"));
  }
  else if (cmd == "RESET") {
    repCount = 0; setCount = 0; holdDuration = 0;
    Serial.println(F("{\"event\":\"reset\"}"));
  }
  else if (cmd.startsWith("TARGET:")) {
    int newTarget = cmd.substring(7).toInt();
    if (newTarget >= 30 && newTarget <= 130) {
      // TARGET_ANGLE is const but we handle via variable in extended version

```

```
    Serial.print(F("{\"event\": \"target_set\", \"angle\": \"\"});
    Serial.print(newTarget);
    Serial.println(F("{}"));
}
}
else if (cmd.startsWith("HR:")) {
    heartRateBPM = cmd.substring(3).toInt();
}
else if (cmd.startsWith("SETS:")) {
    repsPerSet = cmd.substring(5).toInt();
}
}

// ----- HELPERS -----
String stateToString(ExerciseState s) {
    switch (s) {
        case STATE_IDLE:      return "IDLE";
        case STATE_EXTENDED:  return "EXTENDED";
        case STATE_FLEXING:   return "FLEXING";
        case STATE_AT_TARGET: return "AT_TARGET";
        case STATE_HOLDING:   return "HOLDING";
        case STATE_RETURNING: return "RETURNING";
        case STATE_OVERBEND_ALERT: return "OVERBEND";
        case STATE_OVERLOAD_ALERT: return "OVERLOAD";
        default:               return "UNKNOWN";
    }
}
```

